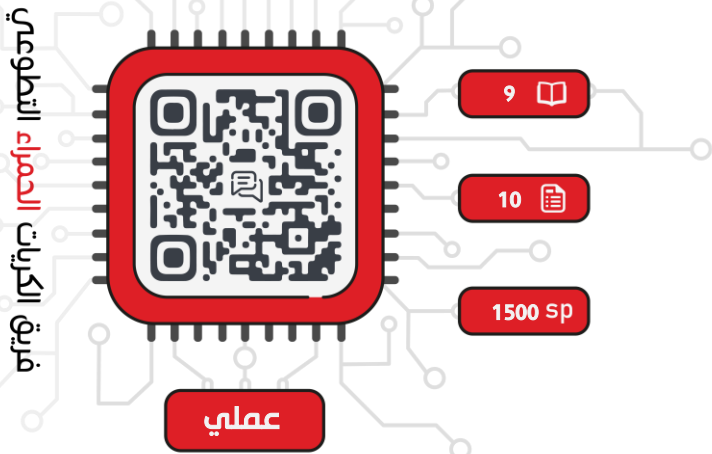




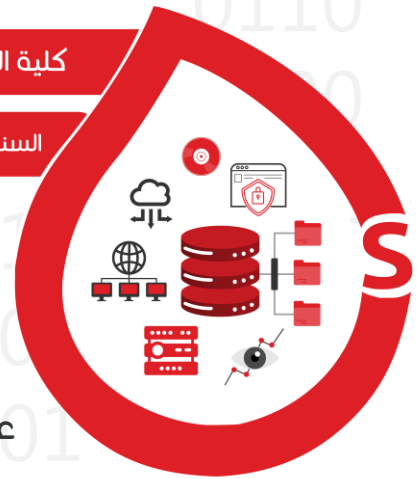
محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 22/12/2025

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Triggers

عملي مشترك



قواعد معطيات ٢

ما هو ال Trigger وما الفرق بينه وبين ال Function أو ال Procedure؟

ال Trigger هو كود SQL يُنفذ تلقائياً عند حدوث حدث معين على جدول أو View فهو لا يُستدعى يدوياً ولا يقبل Parameters، بخلاف ال Function و ال Procedure اللتين يتم استدعاؤهما يدوياً.

سؤال: كيف يتم إنشاء ال Trigger؟

يتم إنشاؤه بشكل مباشر ويكون تابعاً لـ Table.

سؤال: هل لـ Trigger علاقة بنوع أمر DML؟

نعم، لأنه، على سبيل المثال، إذا كان لدينا Trigger تابع لجدول ما ومخصص لأمر Insert، فإنه يُستدعى بمجرد تنفيذ أمر Insert على هذا الجدول. ومنه نستنتج أن هناك علاقة بين ال Trigger ونوع أمر DML المستخدم.

سؤال: كيف نقوم بإنشاء ال Trigger؟

```
CREATE TRIGGER trigger_name
ON table_name
FOR DML (INSERT, UPDATE, DELETE)
AS
BEGIN
-- Trigger body
END
```

يوجد ضمن ال Trigger نوعان من ال (Objects) الافتراضية هما Deleted و Inserted، وبنيتهما (Body) تتطابق مع بنية الجدول (Table) المرتبط به. وهما في الحقيقة جدولان افتراضيان، وبنيتهما متطابقة مع بنية الجدول الذي نتعامل معه. ويمكننا التعامل مع هذين الجدولين، ولكن بطريقة للقراءة فقط (Read-Only) ضمن نطاق ال Trigger.





- سؤال: ما هو محتوى كل من Deleted و Inserted؟
- كل شيء جديد (new value) يتم إضافته يوضع في الجدول Inserted (القيم الجديدة).
- كل شيء عبارة عن حذف أو تحديث لل data يضاف في جدول ال Deleted (القيم القديمة).

ال Deleted و Inserted يمكن أن يحتويان أكثر من سجل.

مثال: لنفترض أن لدينا الجدول التالي T1:

C1 (PK)	C2	C3
1	a	b
2	c	d

الحالة الأولى : عملية INSERT.

لنقم بتنفيذ التعليمة التالية:

```
INSERT INTO T1 (pk, c2, c3)
VALUES (3, 'N', 'F');
```

ال Trigger الخاص بالـ INSERT:

عند تنفيذ العملية أعلاه، يكون محتوى الجدول الافتراضي Inserted هو:

```
SELECT * FROM Inserted;
```

النتيجة:

C1	C2	C3
3	N	F

الحالة الثانية: عملية UPDATE

لنقم بتحديث البيانات بالتعليمة التالية:

```
UPDATE T1
SET c2 = 'aa'
WHERE pk = 1;
```

ال Trigger الخاص بالـ UPDATE:

عند تنفيذ عملية التحديث، يكون محتوى الجدولين الافتراضيين كالتالي:

1. الجدول Inserted (يحتوي على القيم الجديدة بعد التحديث):

```
SELECT * FROM Inserted;
```

C1	C2	C3
1	aa	b

2. الجدول Deleted (يحتوي على القيم القديمة قبل التحديث):

```
SELECT * FROM Deleted;
```

C1	C2	C3
1	a	b

■ سؤال: هل يمكننا إنشاء Trigger على Update و Delete في نفس الوقت؟

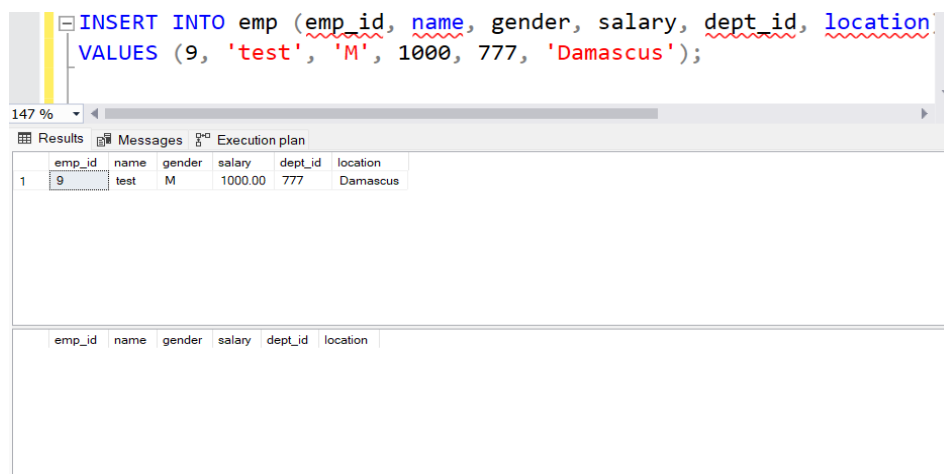
نعم يمكننا ذلك مثال:

لنقم بإنشاء Trigger بسيط:

```
CREATE TRIGGER Test_trigger
ON emp
FOR INSERT
AS
BEGIN
    SELECT * FROM Inserted;
    SELECT * FROM Deleted;
END
GO
```

ملاحظة: قمنا بإنشاء الـ Trigger على عملية INSERT على جدول الموظفين (emp). سيقوم هذا الـ Trigger بتنفيذ عملية SELECT على الجدولين الافتراضيين Inserted و Deleted عند القيام بعملية INSERT على جدول الموظفين. لنقم الآن بتنفيذ عملية إدراج على جدول الموظفين:

```
INSERT INTO emp (emp_id, name, gender, salary, dept_id, location)
VALUES (9, 'test', 'M', 1000, 777, 'Damascus');
```



نلاحظ أنه قام بإرجاع جدولين:

الجدول الأول: يحتوي على السجل الجديد الذي أضفناه، وهو محتوى الجدول Inserted.
الجدول الثاني: (Deleted) فارغ، لأن عملية الإدراج (INSERT) لا تتضمن حذفاً أو تحديثاً لبيانات قديمة.

■ سؤال: إذاً، ما هي فائدة الـ Trigger؟

يسمح لنا بالحصول على القيم الجديدة (new value) والقيم القديمة (old value). وفي حال كنا نريد إنشاء سجل متابعة (Audit Trail)، يساعدنا الـ Trigger في معرفة التغييرات التي طرأت على البيانات، أي أنه يمنع التلاعب بها ويسجل كل شيء.

لنقم الآن بإنشاء جدول تدقيق (Audit Table) لمتابعة جميع العمليات (Transactions) التي تحدث على جدول Dept.

1. أولاً: إنشاء جدول (التدقيق) (Audit Table)

لنقم بإنشاء جدول dbt_dept_audit الذي سيحتوي على id و description لتوثيق كل عملية تتم على جدول Dept.

```
CREATE TABLE dept(
    deptid INT IDENTITY(1,1) PRIMARY KEY,
    dname VARCHAR(50)
);
```

```
CREATE TABLE dbt_dept_audit (
    id INT IDENTITY(1,1) PRIMARY KEY,
    description VARCHAR(255)
);
```

2. ثانياً: إنشاء الـ Trigger

الآن، لنقم بإنشاء الـ Trigger على الجدول الذي نريد متابعته، وهو Dept.

```
CREATE TRIGGER tr_dept_after_insert
ON dept
FOR INSERT
AS
BEGIN
    DECLARE @p_deptid INT;
    DECLARE @p_dname VARCHAR(50);
```

```

SELECT @p_depid = depid, @p_dname = dname
FROM inserted;

INSERT INTO dbt_dept_audit (description)
VALUES (
    'New dept_id ' + CAST(@p_depid AS NVARCHAR(5)) +
    ' with name = ' + @p_dname +
    ' is added at ' + CAST(GETDATE() AS NVARCHAR(20))
);
END
GO

```

شرح:

يقوم هذا الـ Trigger بعمل Insert على جدول الـ Audit مع كل مرة يتم فيها إضافة سجل جديد في جدول الـ Dept. وللحصول على القيم التي يتم تخزينها في الـ Dept، نقوم بتعريف متغيرين (@p_dname, @p_depid) عبر التعليمة Declare، ثم إسناد القيم المخزنة للمتغيرات التي قمنا بتعريفها. لنعرض الآن محتويات كل من جدول dept وجدول التدقيق (Audit):

```

SELECT * FROM dept;
SELECT * FROM dbt_dept_audit;

```

الآن، لنقم بتنفيذ عملية INSERT على جدول dept. سنلاحظ ظهور رسالة (2 row(s) affected). والسبب في ذلك هو أنه في نفس اللحظة التي يتم فيها تخزين القيمة الجديدة في جدول dept، يقوم الـ Trigger بتخزين سجل في جدول التدقيق (Audit).

```
INSERT INTO dept (dname) VALUES ('New Dept');
```

```
(1 row affected)
```

```
(1 row affected)
```

```
Completion time: 2026-01-08T22:34:13.9510304+03:00
```





لنعرض الآن محتويات الجدولين مرة أخرى:

```
SELECT * FROM dept;  
SELECT * FROM dbt_dept_audit;
```

4	4	New dept_id = 4, name = Finance added at Jan 8 2...	2026-01-08 22:34:13.650
---	---	---	-------------------------

الآن، لنقم ببناء الـ Trigger الخاص بعملية الحذف:

```
CREATE TRIGGER tr_tbdept_after_delete  
ON dept  
AFTER DELETE  
AS  
BEGIN  
    DECLARE @p_depid INT;  
    DECLARE @p_dname VARCHAR(50);  
  
    SELECT @p_depid = depid, @p_dname = dname  
    FROM deleted;  
  
    INSERT INTO dbt_dept_audit (description)  
    VALUES (  
        'An existing dept with id = ' + CAST(@p_depid AS NVARCHAR(5)) +  
        ' and name = ' + @p_dname +  
        ' is deleted at ' + CAST(GETDATE() AS NVARCHAR(20))  
    );  
END  
GO
```

الآن، لنقم بحذف السجل الذي أضفناه قبل قليل. سنلاحظ مرة أخرى رسالة (2 row(s) affected).

```
DELETE FROM dept  
WHERE depid = 4;
```



بعد تنفيذ الحذف، سيصبح شكل الجدولين كالتالي:

```
SELECT * FROM dept;
```

```
SELECT * FROM dbt_dept_audit;
```

depid	dname
1	HR
2	IT
3	Finance

id	description
1	New dept_id 1 with name = HR is added at Jan 9 ...
2	New dept_id 2 with name = IT is added at Jan 9 20...
3	New dept_id 3 with name = Finance is added at Ja...
4	New dept_id 4 with name = Helper is added at Jan ...
5	An existing dept with id = 4 and name = Helper is d...

ونلاحظ، أن سجل المتابعة في جدول التدقيق قد وثّق لنا القسم الذي تم حذفه، مما يوفر لنا وسيلة لمعرفة البيانات التي كانت موجودة قبل الحذف.

ملاحظة :

- في كل من ال Insert و Delete نحن نضمن أن التعديل يحصل على سجل واحد فيرد لنا متحول dep_id, dname واحد في كل مرة، فماذا لو تم عمل insert لعدة سجلات في مرة واحدة؟
- عندها عملية الإسناد ستعطي Exception لأن الاستعلام الواحد يرد N Record (مصفوفة من المتحولات) فلا يمكن تخزينها بمتحول واحد.
- إذاً لو كان التغير الحاصل على أكثر من Row كيف نعالج المشكلة السابقة؟
- يكون الحل بتخزين كامل ال values من ال INSERTED في جدول مؤقت TempDept # (وال # ترمز لكونه جدول مؤقت يتم بناؤه ضمن ال Trigger scoop وبمجرد الخروج من ال scoop يتم التخلص من ال ob فلا يعود له وجود)
- تكون بنية هذا الجدول هي نفسها بنية ال INSERTED (أي بنية ال Dept) وبعد تخزين ال INSERTED في هذا الجدول المؤقت، نقوم بعمل حلقة loop: طالما يوجد Data ضمن الجدول المؤقت، قم بعمل select لأول Row من ال Temp (أي من ال INSERTED)
- وبالتزامن معه نقوم بعمل Select من جدول Deleted حيث قيمة ال id مساوية لقيمة ال id الذي تم عمل INSERT له، وبعد قراءة كامل سجلات الجدول Temp نقوم بعمل Delete له حتى لا ندخل ب infinity loop
- وهذه هي الطريقة الصحيحة لعمل Trigger، في حين أن ال Trigger الخاص ب ال Insert و ال Delete السابقين غير صحيحين عملياً.

لنقم الآن ببناء ال Trigger لعملية التحديث (UPDATE) بالطريقة التي ذكرناها سابقاً:

```
CREATE TRIGGER tr_dept_for_update
ON dept
FOR UPDATE
AS
BEGIN
    DECLARE @depid INT;
    DECLARE @old_name NVARCHAR(50), @new_dname VARCHAR(50);
    DECLARE @auditstring VARCHAR(500);

    SELECT *
    INTO #tempdept
    FROM inserted;

    WHILE EXISTS (SELECT 1 FROM #tempdept)
    BEGIN
        SET @auditstring = '';

        SELECT TOP 1
            @depid = depid,
            @new_dname = dname
        FROM #tempdept;

        SELECT @old_name = dname
        FROM deleted
        WHERE depid = @depid;

        SET @auditstring = 'Dept with id = ' + CAST(@depid AS NVARCHAR(50)) +
            ' changed name from ' + @old_name +
            ' to ' + @new_dname;
```

```
INSERT INTO tbdept_audit (description)
VALUES (@auditstring);
```

```
DELETE FROM #tempdept
WHERE deptid = @deptid;
```

```
END
```

```
END
```

```
GO
```

الآن، لنقم بعمل UPDATE لكل قيم dname في جدول dept (التي تؤثر على 3 سجلات):

```
UPDATE dept SET dname = dname + '_updated';
```

نلاحظ أن الـ Trigger قام بتسجيل القيم القديمة مع التحديث الحاصل عليها في جدول الـ Audit.

```
SELECT * FROM dept;
```

```
SELECT * FROM dbt_dept_audit ;
```

	deptid	dname
1	1	HR_updated
2	2	IT_updated
3	3	Finance_updated

	id	description	audit_date
1	1	New dept_id = 3, name = Finance added at Jan 8 2...	2026-01-08 22:33:53.017
2	2	New dept_id = 2, name = IT added at Jan 8 2026 10...	2026-01-08 22:33:53.017
3	3	New dept_id = 1, name = HR added at Jan 8 2026 1...	2026-01-08 22:33:53.017
4	4	New dept_id = 4, name = Finance added at Jan 8 2...	2026-01-08 22:34:13.650
5	5	Dept deleted. id = 4, name = Finance at Jan 8 2026 ...	2026-01-08 22:36:19.873
6	6	Dept id = 3 changed name from Finance to Finance...	2026-01-08 22:37:42.707
7	7	Dept id = 2 changed name from IT to IT_updated at J...	2026-01-08 22:37:42.707
8	8	Dept id = 1 changed name from HR to HR_updated ...	2026-01-08 22:37:42.707

ولو لم نعمل حلقة تكرارية (Loop)، لما تمكنا من تخزين القيم الثلاث للتحديثات التي حصلت في نفس الوقت، ولكن الـ Trigger أعطى خطأ (Exception).

▪ لحذف Trigger:

```
DROP TRIGGER TriggerName
```

1. Trigger ال هو كود SQL ينفذ تلقائياً عند حدوث حدث معين على جدول أو View.

2. متى يعمل ال Trigger؟

يعمل بشكل تلقائي، أي لا يمكن تشغيله يدوياً.

3. متى نقوم باستخدامه؟؟؟

_التحقق من البيانات

_الحماية من حذف أو تعديل غير مسموح

_تسجيل عمليات ال logs

_منع عمليات غير مسموح بها

4. ال Functions:

ترجع قيمة واحدة أو جدولاً.

5. متى نقوم باستخدامها؟

_داخل Select

_داخل Where

_داخل ال Join

ويجب أن ترجع قيمة.

6. ال Procedures:

مجموعة اوامر SQL تنفذ عند استدعائها.

يمكنها تعديل البيانات،

يمكن ان يرجع اكثر من قيمة ويمكن تمرير

parameters إليها

ولا يمكن استخدامها مع ال Select مباشرة.

7. اي ان ال:

Trigger يحدث تلقائياً

Function يرجع قيمة

Procedure ينفذ مجموعة اوامر

8. في ال Trigger، يوجد جدولان مؤقتان هما:

Inserte الذي يحوي على البيانات الجديدة التي

تم ادخالها

Delete يحوي البيانات القديمة قبل الحذف او

التحديث

9. حذف ال Trigger:

Drop Trigger TriggerName

10. ال Trigger لا يُستدعى بـ Execute ولا يأخذ معلمات (Parameters)، ويعمل على صفوف متعددة.

11. ال Triggers في SQL هي على مستوى التعليمة (Statement-Level) وليست على مستوى الصف (Row-Level).

انتهى المقرر

إذا لم يكن لديك حلم فستكون جزء من أحلام الآخرين